



> Menu

Overview

Relevant source files

Purpose and Scope

This document provides a high-level introduction to the OpenFinRAM ASAP7 SRAM Compiler, explaining its purpose, architecture, and core components. It is designed to help new users and developers understand what the system does and how its pieces fit together.

For detailed build instructions, see [Getting Started](#). For in-depth architectural details, see [Architecture](#). For API usage, see [API Reference](#).

What is the ASAP7 SRAM Compiler?

The OpenFinRAM ASAP7 SRAM Compiler is a command-line tool that generates Static Random-Access Memory (SRAM) layouts targeting the ASAP7 7nm Process Design Kit (PDK). The compiler reads a GDSII cell library file containing basic SRAM building blocks and is intended to produce a complete SRAM array layout with the specified number of rows and columns.

The tool leverages two key external libraries:

- **GDSTK**: For reading, manipulating, and writing GDSII layout files
- **PLOG**: For structured logging and debugging

Sources: [README.md](#) | 1-9 [src/main.cpp](#) | 26-31

Key Components

The system consists of three primary layers:

Importa

Ask Devin about shao-chien-lu/OpenFinRAM

 Fast 



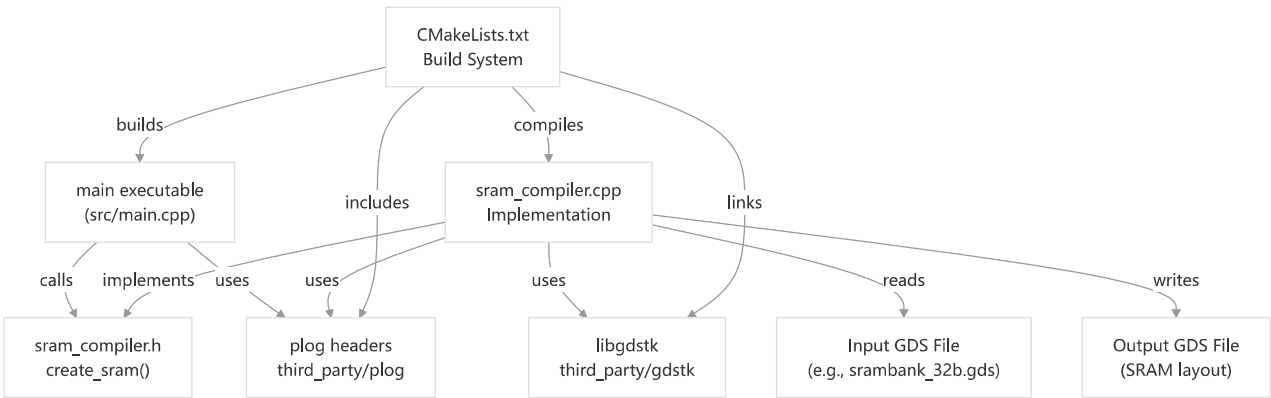
Layer	Component	File	Importance	Role
Core Logic	SRAM Compiler	src/sram_compiler.cpp	2.0	GDS I/O coordination, library management
Core Logic	API Header	include/sram_compiler.h	2.0	Public interface definition
External	GDSTK	third_party/gdstk	-	GDS file operations
External	PLOG	third_party/plog	-	Logging infrastructure
Build	CMake Configuration	CMakeLists.txt	4.0	Build orchestration, dependency linking

Sources: CMakeLists.txt | 1-24 src/main.cpp | 1-45

System Architecture

The following diagram illustrates the three-tier architecture and how components interact:

Three-Tier Architecture



Sources: CMakeLists.txt | 1-24 src/main.cpp | 1-45

Command-Line Interface

The entry point `main()` function in `src/main.cpp` provides the user-facing interface. It expects four

Ask Devin about shao-chien-lu/OpenFinRAM



3. **Number of columns:** Unsigned integer specifying array columns
4. **Output GDS file path:** Path where the generated SRAM layout will be written

The CLI performs the following operations:

- Initializes the `plog` logging system with debug-level output `src/main.cpp` | 10-12
- Validates command-line argument count `src/main.cpp` | 14-18
- Parses and logs input parameters `src/main.cpp` | 20-37
- Invokes `create_sram()` from the core API `src/main.cpp` | 39-40
- Handles error codes and returns appropriate exit status `src/main.cpp` | 41-44

Sources: `src/main.cpp` | 9-45

Core API

The public interface is defined in `include/sram_compiler.h` and consists of a single function:

```
gdstk::ErrorCode create_sram(
    const char* input_gds_file,
    unsigned num_rows,
    unsigned num_cols,
    const char* output_gds_file
);
```

This function signature establishes the contract between the CLI layer and the compilation logic. The implementation resides in `src/sram_compiler.cpp` and uses `gdstk::ErrorCode` for error reporting.

For detailed API documentation, see [create_sram Function](#).

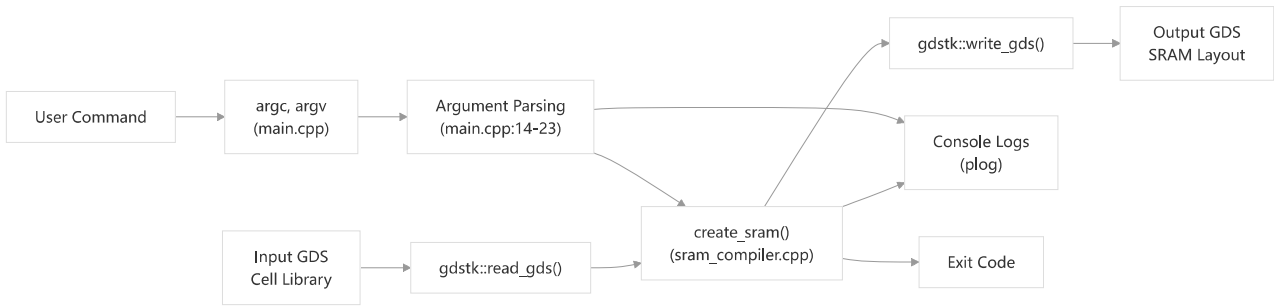
Sources: `src/main.cpp` | 7 `src/main.cpp` | 39-40

Data Flow

The following diagram shows how data transforms through the system from user input to GDS

Ask Devin about shao-chien-lu/OpenFinRAM





Sources: `src/main.cpp` | 9-45 `CMakeLists.txt` | 14-17

External Dependencies

The system integrates two third-party libraries via Git submodules:

GDSTK (GDS Toolkit)

- **Purpose:** Reading and writing GDSII layout files
- **Location:** `third_party/gdstk`
- **Integration:** Compiled as a static library and linked into the main executable
- **Build:** Handled via CMake's `add_subdirectory()` directive `CMakeLists.txt` | 7
- **Usage:** Provides `gdstk::read_gds()` and `gdstk::write_gds()` functions

PLOG (Portable Logging Library)

- **Purpose:** Structured console logging with severity levels
- **Location:** `third_party/plog/include`
- **Integration:** Header-only library, included via CMake `CMakeLists.txt` | 11
- **Initialization:** `plog::ColorConsoleAppender` with `plog::TxtFormatter` `src/main.cpp` | 11-12
- **Macros Used:** `LOGI` (info), `LOGD` (debug), `LOGE` (error)

For more details on dependency management, see [External Dependencies](#).

Sources: `CMakeLists.txt` | 6-11 `src/main.cpp` | 1-12 `README.md` | 5-9

Ask Devin about shao-chien-lu/OpenFinRAM



Setting	Value	Purpose
Minimum CMake Version	3.22	Compatibility baseline
Project Name	ASAP7_PDK_SRAM_Compiler	Build identifier
C++ Standard	C++11	Language standard
Executable Name	main	Output binary name
Source Files	src/main.cpp , src/sram_compiler.cpp	Compilation units
Include Directories	include/ , third_party/plog/include/	Header search paths
Linked Libraries	gdstk	Static library dependencies

The build process compiles two source files, links against the GDSTK library, and produces a single executable named `main`.

For detailed build instructions, see [Installation and Build](#). For CMake details, see [CMake Configuration](#).

Sources: CMakeLists.txt | 1-24

Current Implementation Status

 **Important Limitation:** The current implementation provides the infrastructure for SRAM compilation but lacks the actual layout generation logic. Specifically:

Implemented:

-  Command-line argument parsing and validation
-  Logging infrastructure with severity levels
-  GDS file I/O operations via GDSTK
-  Error handling and status reporting
-  Build system configuration with dependency management

Not Yet Implemented:

-  SRAM cell array placement and routing
-  Decoder and peripheral circuit generation

Ask Devin about shao-chien-lu/OpenFinRAM



The `create_sram()` function creates an empty GDS library structure but does not populate it with the actual memory array layout. This represents a significant implementation gap that future development must address.

For a detailed analysis of what needs to be implemented, see [Implementation Status](#).

Sources: Analysis based on system architecture diagrams and code structure

Technology Context

The compiler targets the **ASAP7 7nm Process Design Kit**, an open-source, academic PDK developed by Arizona State University in collaboration with ARM. This PDK provides:

- Design rules for 7nm FinFET technology
- Standard cell libraries
- Timing and power models
- Physical layout constraints

The input GDS files (such as `srambank_32b.gds`) contain pre-designed SRAM cell layouts that comply with ASAP7 design rules. The compiler's role is to instantiate and arrange these cells into larger memory arrays.

For detailed information about ASAP7, see [ASAP7 Process Design Kit](#). For GDS file format details, see [GDS Cell Libraries](#).

Sources: `README.md` | 7

Licensing

The project is licensed under the **BSD 3-Clause License**, which permits:

- Commercial and non-commercial use
- Modification and redistribution
- Both source and binary distribution

The license requires:

- Retention of copyright notices

Ask Devin about shao-chien-lu/OpenFinRAM



For complete license terms, see [Licensing](#).

Sources: [LICENSE](#) | [1-29](#) | [README.md](#) | [11-12](#)

Getting Started

To begin using the ASAP7 SRAM Compiler:

- 1. **Build the project:** Follow the instructions in [Installation and Build](#)
- 2. **Run your first compilation:** See [Running the Compiler](#) for command-line usage
- 3. **Understand the architecture:** Review [Architecture](#) for implementation details
- 4. **Explore the API:** See [API Reference](#) for programmatic usage

For more information, see the [FAQ](#) or [Contact Us](#) page.

Ask Devin about shao-chien-lu/OpenFinRAM

